1

DBMS

(Database Management System)

is based on

defines

Data Model

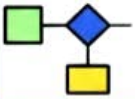
(Defines how data is organized and related)

is represented using

Schema

(Logical structure of the database)

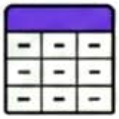
describes the structure of the database



ER Diagram

(Graphical representation of data model)

consists of



Instance

(Actual data stored at a particular time)

represents the data at a particular time

Entity

(A real world object)

represents a real world object

Attribute

(Property or characteristic of an entity)

describes the properties of an entity

Relationship

(Association between two or more entities)

represents an association between entities

Flow Summary : DBMS → Data Model → (Schema → Instance)

and Data Model → ER Diagram → (Entity, Attribute, Relationship)

ANSI/SPARC Three-Level Architecture

2

User 1

User 2

User n

External Level

External View 1 (View of User 1)

External View 2 (View of User 2)

• • •

External View n (View of User n)

- Describes how each user views the data.
- Multiple external views for different users/applications.

External / Conceptual Data Independence

Changes in the conceptual schema do not affect the external views.

Conceptual Level

Conceptual Schema (Community View)

- Describes the whole database logically.
- Hides storage details from users.

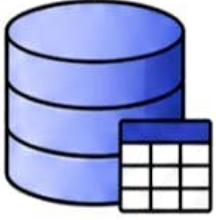
Conceptual / Internal Data Independence

Changes in the internal schema do not affect the conceptual schema.

Internal Level

Internal Schema (Physical View)

- Describes how data is stored physically.
- Includes access paths, file organization, indexes etc.



Stored Database (Physical Data)

3

File-Based Systems vs Database Systems

FILE-BASED SYSTEMS

Data is stored in separate files owned by individual applications.

Both store and manage data used by applications

DATABASE SYSTEMS (DBMS)

Data is stored centrally and managed by a DBMS.

PROBLEMS / LIMITATIONS

Data Redundancy & Inconsistency
Same data is stored in multiple files. Updates may not be reflected everywhere.

Data Isolation
Data is scattered in different files and formats; difficult to access.

Program–Data Dependence
Data format is embedded in programs. Any change requires program changes.

Lack of Data Integrity
No effective way to enforce rules and constraints on data.

Poor Security
Access control is hard to implement consistently.

Difficult Data Sharing
Sharing data among applications is complex and error-prone.

Hard to Maintain & Extend
High maintenance cost; difficult to adapt to new requirements.

DBMS

(Database Management System)

A centralized system that manages data efficiently and provides an environment for data storage, retrieval and manipulation.

SOLUTIONS PROVIDED BY DBMS

Controls Redundancy & Ensures Consistency
Centralized data and integrity constraints keep data consistent.

Data Integration
All data is stored in a unified structure, easy to access.

Data Independence
Logical and physical data independence separate data from programs.

Enforces Data Integrity
Constraints and rules ensure accurate and valid data.

Better Security
Centralized authorization and access control.

Easy Data Sharing
Data can be shared safely among multiple users and applications.

Easier Maintenance & Scalability
Centralized control makes maintenance easier and system scalable.

OVERALL BENEFITS OF DBMS

- Improved Data Accuracy
- Enhanced Security
- Better Data Sharing
- Reduced Redundancy & Cost
- Improved Decision Making
- Increased Productivity

4

ER DIAGRAM COMPONENTS
(Concept Map)

1. ENTITY

An entity is a distinguishable object or concept in the real world about which data is stored.

Notation

Rectangle

Example

STUDENT

Key Points

- Has an independent existence.
- Represented by a rectangle.
- Each entity has properties (attributes).

2. WEAK ENTITY

A weak entity does not have a primary key of its own and depends on another entity (Owner entity) for its identity.

Notation

Double Rectangle

Example

DEPENDENT (weak entity)

Key Points

- Cannot exist without its owner entity.
- Identified using partial key (discriminator).
- Represented by double rectangle.

3. ATTRIBUTE TYPES

Attributes describe the properties or characteristics of an entity or relationship.

Types of Attributes

1. **Simple Attribute** — Atomic, indivisible. (Age)
2. **Composite Attribute** — Can be divided into subparts. (Address → Street, City, Pin)
3. **Single-valued Attribute** — Has one value for an entity. (Email)
4. **Multi-valued Attribute** — Has multiple values for an entity. (Phone_No)
5. **Derived Attribute** — Can be derived from other attribute(s). (Age (from DOB))

Key Point

Attributes are represented by ovals. Different types are shown using different notations.

4. RELATIONSHIP TYPES

A relationship is an association among two or more entities.

Notation

R — Diamond

Types

1. One-to-One (1:1) — A — R — B
   (Each A is related to one B and vice versa)
2. One-to-Many (1:N) — A — R — B
   (One A is related to many B's)
3. Many-to-One (N:1) — A — R — B
   (Many A's are related to one B)
4. Many-to-Many (M:N) — A — R — B
   (Many A's are related to many B's)

Key Point

Relationships are represented by diamonds and have cardinality (1:1, 1:N, N:1, M:N).

5. PARTICIPATION CONSTRAINTS

It specifies whether the participation of an entity in a relationship is mandatory or optional.

Types

1. **Total Participation (Mandatory)**
   Every entity must participate in the relationship.
   A — R — B
2. **Partial Participation (Optional)**
   Some entities may or may not participate.
   A — R — B

Key Point

Total participation is shown by double line; partial participation by single line.

LEGEND

Entity · Weak Entity · Attribute · Multi-valued Attribute · Derived Attribute · Relationship · Partial Participation · Total Participation

5

COMPARISON OF DATA MODELS
(Concept Map)

DATA MODELS
Define how data is organized, related and manipulated.

1. HIERARCHICAL MODEL
Data is organized in a tree structure with a single root. Each child has only one parent.

2. NETWORK MODEL
Data is organized as a graph structure. A record (child) can have multiple parents.

3. RELATIONAL MODEL
Data is organized in tables (relations) of rows and columns. Relationships are established using keys.

	1. HIERARCHICAL MODEL	2. NETWORK MODEL	3. RELATIONAL MODEL
STRUCTURE	Tree structure (one-to-many)	Graph structure (many-to-many)	Table structure (relations)
RELATIONSHIP	Each child has only one parent.	A child can have multiple parents.	Relationships via primary and foreign keys.
DATA ACCESS	Navigation based (top-down traversal from root).	Navigation based using sets and links.	Non-navigational; uses SQL for high-level access.
INTEGRITY	Limited integrity constraints.	More integrity than hierarchical but complex.	Strong integrity constraints (keys, domains, referential integrity).
FLEXIBILITY	Rigid structure; hard to modify.	More flexible than hierarchical but complex to design.	Highly flexible; easy to modify and maintain.
SCALABILITY	Not suitable for large and complex data.	Better than hierarchical; can handle complex data.	Highly scalable; suitable for large databases.
EXAMPLES	IBM Information Management System (IMS)	CODASYL DBTG, IDMS	MySQL, Oracle, SQL Server, PostgreSQL, DB2

KEY TAKEAWAY
Hierarchical = Simplicity (one parent) • Network = Power (many parents) • Relational = Simplicity + Flexibility + Integrity

6

EER MODEL EXTENSIONS

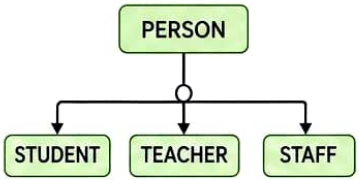
EER Model
(Enhanced ER Model)

extends ER model with

GENERALIZATION

Combines multiple entity types into a supertype.

Top-Down Approach

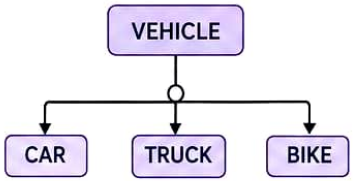


Example:
Person is a supertype of Student, Teacher, and Staff.

SPECIALIZATION

Divides a supertype into multiple subtype.

Bottom-Up Approach

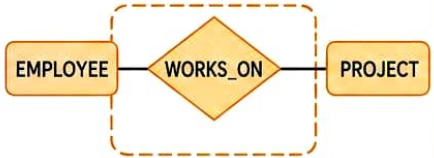


Example:
Vehicle is specialized into Car, Truck, and Bike.

AGGREGATION

Treats a relationship set as a higher-level entity.

Abstracts a relationship for further use.



Example:
WORKS_ON relationship is treated as an entity for further relationships.

These extensions improve the modeling power of ER model to represent real-world scenarios more effectively.

7 DBMS SOFTWARE ARCHITECTURE

Flow from User Request to Data Retrieval

USER

User Request

Data Retrieval

1 USER INTERFACE
User issues a query or request through an application.

Applications provide forms, menus or query interfaces to interact with the DBMS.

2 APPLICATION PROGRAM
Sends the request to the DBMS through an interface (API).

Application program communicates with DBMS using SQL or embedded SQL.

3 QUERY PROCESSOR
Parses, validates and translates the query. Generates an efficient execution plan.

Syntax check, semantic check, optimization and plan generation are performed here.

4 QUERY OPTIMIZER
Evaluates different execution plans and chooses the best one.

Cost is estimated for each plan based on statistics, and the best plan is selected.

5 EXECUTION ENGINE
Executes the chosen plan and requests data from storage.

Performs operations (scan, join, sort, etc.) as per the execution plan.

6 STORAGE MANAGER
Accesses data from disk using file and index structures.

Manages buffer, file access, index traversal and disk space management.

7 DATA
Retrieved data is sent back through the same path to the user.

Result is returned to the user via application.

ARCHITECTURE SUMMARY

- DBMS acts as an interface between users/applications and the physical database.
- Each component has a specific role in processing the request efficiently.
- The architecture ensures data independence, security, concurrency control and efficient retrieval.

KEY BENEFITS

Security & Integrity
Ensures data security and integrity.

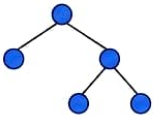
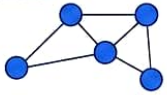
Efficiency
Optimized query processing for faster results.

Data Independence
Logical and physical data independence.

Concurrency Control
Handles multiple users accessing data simultaneously.

8 DATA MODELS AND REAL-WORLD APPLICATIONS

Mapping data models to practical use and their suitability

DATA MODEL	DESCRIPTION	REAL-WORLD APPLICATIONS	SUITABILITY	KEY FEATURES
1. HIERARCHICAL MODEL 	• Data is organized in a tree-like structure. • Each child has only one parent.	File systems (e.g., Windows Explorer) Organization charts XML data handling	✓ Suitable for data with strict parent-child relationships. ✗ Less flexible for many-to-many relationships.	• Simple and efficient for hierarchical data. • Fast access. • Limited flexibility.
2. NETWORK MODEL 	• Data is organized as a graph with many-to-many relationships. • Records can have multiple parents.	Telecommunication networks Airline reservation systems Supply chain management	✓ Efficient for complex many-to-many relationships. ✗ Complex to design and maintain.	• Supports complex relationships. • Efficient data navigation. • More complex than hierarchical.
3. RELATIONAL MODEL 	• Data is represented in tables (relations) of rows and columns. • Relationships are established using keys.	Banking systems E-commerce platforms University/College systems	✓ Highly flexible, easy to use and widely supported. Ideal for most business applications.	• Simple and intuitive. • Strong data integrity. • Uses SQL. • Scalable and robust.
4. ENTITY-RELATIONSHIP MODEL 	• Conceptual model using entities, attributes and relationships. • Used for designing databases.	Database design System analysis Data modeling and documentation	✓ Best for conceptual design and communication. Not directly used for data storage.	• Easy to understand. • Good visualization. • Basis for converting to other models.
5. OBJECT-ORIENTED MODEL 	• Data is represented as objects with attributes and methods. • Supports inheritance, encapsulation, etc.	CAD/CAM systems Multimedia applications AI and Simulation systems	✓ Suitable for complex data like images, videos, CAD, etc. ✗ Less standardization compared to relational.	• Supports complex data types. • Inheritance and polymorphism. • Good for object-oriented applications.

Summary : Choose the data model based on the nature of data, relationships, performance needs, and application requirements.

9

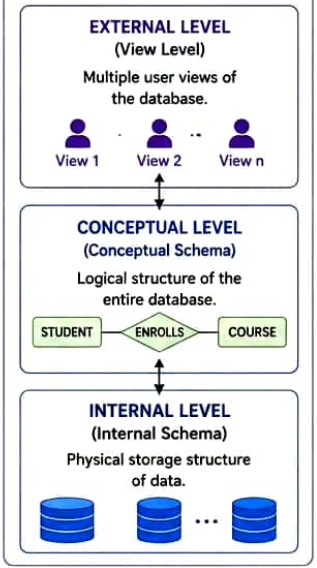
SCHEMAS AND INSTANCES IN DBMS

Relationship between Physical and Logical Data Independence

KEY CONCEPTS

- **Schema:** The blueprint or structure of the database at a particular level.
- **Instance:** The actual data stored in the database at a particular time.

THREE LEVEL ARCHITECTURE

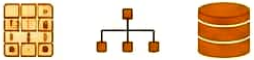


SCHEMAS AND INSTANCES

SCHEMA (Structure)	INSTANCE (Data)
EXTERNAL SCHEMA (View Schema) Defines how a user sees the data.	**EXTERNAL INSTANCE (View Instance)** Actual data as seen by a user.
CONCEPTUAL SCHEMA (Conceptual Level) Defines the logical structure of the whole database.	**CONCEPTUAL INSTANCE (Conceptual Level)** Actual data in the logical structure.
INTERNAL SCHEMA (Internal Level) Defines how data is stored physically.	**INTERNAL INSTANCE (Internal Level)** Actual data in the physical storage.

Roll No	Name	Course
101	Ramesh	DBMS
102	Sita	DBMS
...		

Roll No	Name	Course
101	Ramesh	DBMS
102	Sita	DBMS
103	John	OS
...		

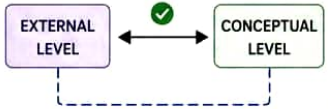


```
1010101010101010
0101001101010101
1010101110101001
...
```

DATA INDEPENDENCE

1. LOGICAL DATA INDEPENDENCE

Changes in the conceptual schema do not affect external schemas and user views.

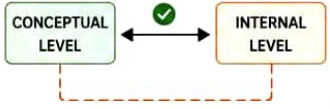


Shielded from changes in conceptual schema.

Example:
Adding a new attribute in the conceptual schema does not affect user views.

2. PHYSICAL DATA INDEPENDENCE

Changes in the internal schema do not affect the conceptual schema.



Shielded from changes in internal schema.

Example:
Changing file organization or adding an index does not affect the logical structure.

SUMMARY Schemas describe the structure at different levels, while instances represent the actual data at that time. Data independence ensures changes at one level do not impact other levels, making the DBMS flexible and robust.

10 ROLES AND RESPONSIBILITIES IN A DBMS ENVIRONMENT

DBA, End User, Application Developer

1. DATABASE ADMINISTRATOR (DBA)

Overall control of the database system and responsible for its proper functioning.

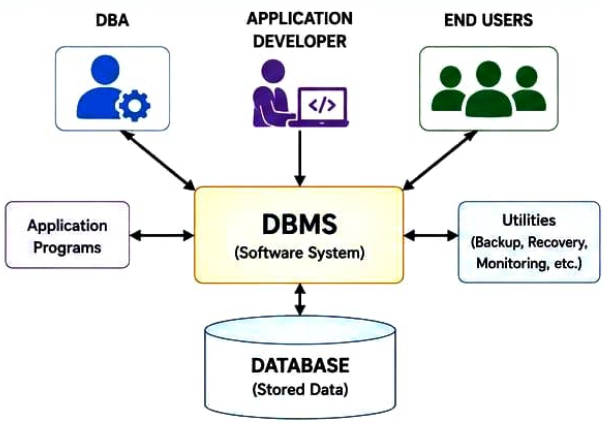
Responsibilities

- Define database schema and storage structures.
- Create and manage user accounts and authorization.
- Ensure data security and integrity.
- Handle backup and recovery.
- Monitor performance and tune the system.
- Manage concurrency control and transactions.
- Ensure availability and reliability of the database.
- Handle hardware and software environment.

Example

DBA creates tables, sets permissions, performs backup, and ensures the system runs smoothly.

DBMS ENVIRONMENT



FLOW EXPLANATION

- Direct interaction
- Supports / Uses

All roles interact with the DBMS which manages the database and provides services to users.

2. END USER

Users who access the database through applications to perform various tasks.

Responsibilities

- Use applications to perform queries and transactions.
- Enter, view, update and delete data as per permissions.
- Generate reports and retrieve information.
- Follow data policies and security rules.
- Provide requirements and feedback for improvements.

Example

A student uses the application to view marks, a bank employee updates customer details, a salesperson generates reports.

3. APPLICATION DEVELOPER

Designs and develops applications that interact with the database.

Responsibilities

- Analyze user requirements and design application logic.
- Write programs and queries to access the database.
- Design user interfaces and reports.
- Test and debug applications.
- Ensure efficient interaction between application and DBMS.

Example

A developer creates a payroll system application that allows employees to update information and generate salary reports from the database.

KEY POINTS

- All roles are essential for the effective functioning of a DBMS.
- DBA ensures security, integrity and availability.
- Developers build applications to access data.
- End users use the data for their daily tasks.
- Together, they make the database system reliable, efficient and useful.

SUMMARY

DBA manages the system, **Developers** build the applications, and **End Users** use the data. The DBMS acts as an intermediary that ensures secure, efficient and reliable data management.